PlantDx

The Complete Engineering & Research History

Knowledge-grounded Vision–Language models for leaf-disease diagnosis

A combined engineering handbook, research notebook, architecture document, experiment log, and retrospective — reconstructed entirely from the repository, its git history, and its documentation. Every material claim is traceable to evidence.

Repository: github.com/iAakash1/experimentation (public)
Author of record: iAakash1
History span: 2026-07-11 → 2026-07-17 (16 commits, 12 tags)
Trained models: Qwen2.5-VL-7B-Instruct-4bit · QLoRA · MLX (Apple Silicon)
Crops: Tomato (10 classes) · Mango (8 classes)
Document compiled: 2026-07-18

Preface — How to read this document, and how it was sourced

This is not a README, a paper, or a marketing page. It is the internal engineering and research handbook for **PlantDx** — the document you would open five years from now to remember *why* each decision was made, what was tried, what failed, and what was learned. It is written in the first-person voice of the engineering team, but every material claim is grounded in an artifact you can open and verify.

The evidence discipline

Throughout, important statements carry an Evidence box pointing to the commit, file, configuration, report, or test that supports them. Where the evidence is thin or absent, the document says so explicitly rather than guessing. This discipline mirrors a rule the project imposed on itself and is quoted verbatim later: "*Verify with real tools, not reasoning.*"

EVIDENCE · PRIMARY SOURCES FOR THIS HANDBOOK

Git history — 16 commits (`12efddd` … `9f08d99`), 12 tags, 2 branches (`main`, `feat/mango-training`). **In-repo narrative:** `CLAUDE.md` (a detailed, gitignored engineering-memory file), `CHANGELOG.md`, `PROJECT_CONTEXT.md`, the `docs/` tree, and the frozen design specifications in `caption_framework/` (8 docs) and `ontology_design/` (13 docs). **Ground-truth artifacts:** the Disease Knowledge Base (`knowledge_base/dkb.json`), configs (`configs/`), evaluation reports (`reports/<run>/evaluation/`), and adapter checkpoints.

HONEST GAPS — WHAT THE EVIDENCE DOES NOT CONTAIN

GitHub collaboration artifacts are empty. Verified via the authenticated GitHub CLI on 2026-07-18: **0 issues, 0 pull requests, 0 published releases**, and no discussions or project boards. The repository was developed linearly on `main` by a single author, with one short-lived feature branch (`feat/mango-training`). Therefore there is *no* issue/PR/review/discussion history to mine for rationale — the "why" comes from the specs, the CHANGELOG, the engineering-memory file, and the code itself.

Early commit messages are single-line. Commits `12efddd` through `673ed77` carry only a subject (no body). The rich per-milestone rationale for those was written into `CHANGELOG.md` and `CLAUDE.md` instead, which this document treats as the authoritative narrative for that period. Only the final three commits carry detailed bodies.

Where a founding claim (e.g. "zero-shot VLMs are unreliable") is asserted in the specs but not backed by a standalone benchmark artifact, the document marks it as *stated motivation* and points to the concrete measurement that does exist (the base-model evaluation).

A note on authorship and voice

The three most recent commits record `Co-Authored-By: Claude Opus 4.8`; the project was built with AI-assisted engineering. This handbook is nonetheless written as a first-person engineering record because that is its purpose — to preserve reasoning. No human anecdote, meeting, or conversation is invented; every "we decided" is backed by a decision that is visible in the artifacts.

READING PATH

Sections 1–4 give the arc (vision, timeline, research and repository evolution). Sections 5–10 are the technical core (architecture, dataset, captions, models, training, evaluation). Sections 11–14 are the "war stories" (engineering and research challenges, refactors, performance). Sections 15–18 are reflective (lessons, counterfactuals, interview appendix, retrospective). Section 19 — the **Research Decision Log** — is the structured spine: every major decision as Question → Hypothesis → Experiment → Result → Conclusion → Impact → Evidence.

Table of Contents

Preface — How to read this document 2
1 · Project Vision 5
2 · Complete Timeline 7
3 · Research Evolution 9
4 · Repository Evolution 12
5 · Complete Architecture 14
6 · Dataset Story 18
7 · Caption Evolution 20
8 · Model Evolution 22
9 · Training Evolution 24
10 · Evaluation Evolution 26
11 · Engineering Challenges 30
12 · Research Challenges 33
13 · Major Refactors 35
14 · Performance Improvements 37
15 · Lessons Learned 38
16 · If We Started Again Today 40
17 · Interview Appendix 41
18 · Complete Retrospective 43
19 · Research Decision Log 45
Appendix — Evidence index & glossary 51

1 · Project Vision

1.1 Why PlantDx was started

PlantDx exists to answer one uncomfortable question honestly: *if a general-purpose vision–language model cannot reliably diagnose a crop disease from a leaf photo, can it be trusted to **describe** one?* The project's founding position is that it cannot — and that using such a model to author training captions would launder its errors into any model trained on those captions. So PlantDx removes models from the caption-authoring path entirely and builds the training supervision from a curated, cited knowledge base instead.

FOUNDING THESIS

The differentiator was never "more data." It was that **every caption is traceable, by construction, to a cited scientific source**, that **no caption can assert a symptom not visible in a single-leaf photo** (the observability invariant), and that **the entire dataset can be rebuilt byte-for-byte** from a frozen knowledge base, a frozen ontology, and a seed — a property no LLM/VLM-generated corpus can offer.

1.2 The problem and the gap

Agricultural VLM datasets are typically built one of three ways: (a) hand-written rule-based templates, (b) captions generated by prompting an LLM, or (c) captions generated by prompting a VLM on the training images themselves. Each embeds a failure mode — (a) is rigid and shallow, (b) hallucinates plausible-sounding but unlicensed claims, and (c) is circular (the student learns the teacher's mistakes). The gap PlantDx targets is a **method** for constructing caption supervision that is simultaneously diverse, scientifically grounded, and mechanically reproducible.

EVIDENCE

The three-way comparison and the "circular failure mode" argument are stated in the frozen methodology spec and the paper-contribution docs: `caption_framework/07_ieee_methodology_section.md`, `ontology_design/13_paper_contribution.md`. The base-model numbers that make the argument concrete are the measured zero-shot accuracies in §1.3.

1.3 The empirical fact that anchors every decision

The single measurement that grounds the whole architecture is the base model's zero-shot accuracy on the two held-out test splits. It is poor — near random for the class counts involved.

Crop	Test images	Base (zero-shot) accuracy	Base macro-F1
Tomato	910	7.3%	0.034
Mango	200	17.5%	0.128

EVIDENCE

Read directly from `reports/qwen25vl_tomato_qlora/evaluation/metrics.json` and `reports/qwen25vl_mango_qlora/evaluation/metrics.json` (the `base.classification` block). These are the un-tuned `Qwen2.5-VL-7B-Instruct-4bit` results on the frozen test splits.

EVIDENCE LIMIT — THE "PRE-PROJECT MULTI-MODEL BENCHMARK"

`CLAUDE.md` narrates that "before any of the current architecture was built, the project ran a zero-shot benchmark of general-purpose open-weight VLMs … they performed poorly," and a stub `ZeroShotEvaluator` (`src/plantdx/evaluation/zero_shot.py`) was scaffolded to run base VLMs to "establish the zero-shot baseline that motivates the knowledge-grounded approach." However, **no standalone benchmark artifact with recorded numbers for multiple named models is present in the repository.** The concrete, reproducible evidence for the "VLMs are unreliable" claim is the single-model base evaluation above. The broader four-model zero-shot comparison the `ZeroShotEvaluator` was designed for was descoped (see §8, §10).

1.4 The original vision, and how it evolved

The original vision was a **reproducible artifact, not a claim**: a released, versioned pipeline (knowledge base + ontology + framework + implementation) that other agricultural-VLM researchers could adopt and extend to new crops without redesigning anything, plus an IEEE paper documenting the method. Over the seven-day build the vision sharpened in three ways:

- **From "a dataset" to "a compiler."** Early on the goal was a caption dataset; it became a *chain of deterministic compilers* (`Ontology = f(DKB)`, `Vocabulary = f(Ontology)`, `Corpus = g(Ontology, Vocabulary, Templates, seed)`) where reproducibility is a mechanically-checked property, not an aspiration.
- **From single-crop to multi-crop.** Tomato shipped first; mango was added and — critically — the evaluation was *generalized* so crop is derived from data, never hardcoded (§10, §12).
- **From pipeline to product.** A Streamlit demo was added so the trained adapters are usable and their behavior (confidence, adapter attachment, evaluation) is visible and honest.

1.5 Current vision and future vision

Current: an end-to-end, deterministic, crop-agnostic pipeline that turns a cited knowledge base into instruction-tuning captions, QLoRA-fine-tunes Qwen2.5-VL on Apple Silicon for tomato and mango, evaluates against the base model with official metrics, and serves the result in a demo — with CI (`ruff` / `mypy` / `pytest`) green throughout.

Future: the image-grounded Instruction Dataset Builder and the remaining three declared target models (Qwen3-VL, InternVL3, Gemma-3), plus a per-image diversity engine — all scaffolded but not yet implemented (§4, §8). The north star remains: *make "the captions are grounded and the dataset is reproducible" a property of the software, not a sentence in a paper.*

2 · Complete Timeline

PlantDx was built in an intense, mostly-linear burst: the repository was created at **2026-07-11 12:12 UTC** and reached its public-release state on **2026-07-17**. Sixteen commits, twelve tags. The tags themselves tell a story — they begin as milestone labels (`v0.1-scaffold`, `v0.4-ontology`) and shift mid-project to semantic versions (`v0.1.0-ci-green`, `v0.2.0`, `v1.0.0`), reflecting a transition from "internal milestone" thinking to "release" thinking.

2.1 The build at a glance

Date	Commit	Tag(s)	Milestone	Δ (files / +ins)
07-11	`12efddd`	v0.1-scaffold	Repository scaffold (all stages stubbed)	162 / 8,880
07-11	`3e73d0f`	v0.3-normalization	Dataset Audit + Normalization engines	42 / 2,289
07-12	`2125fc9`	v0.4-ontology	Deterministic ontology compiler	34 / 3,244
07-12	`584f49e`	v0.5-stabilization	CI/style stabilization + dev docs	86 / 1,611
07-12	`7b45bb2`	v0.5-vocabulary	Vocabulary + symptom lexicon compiler	27 / 1,964
07-13	`05a98e1`	v0.6-corpus-freeze, v0.6-generation	Concept models, templates, caption corpus, exporters	58 / 5,200
07-13	`8ad2726`	v0.8-tomato-training	Tomato QLoRA training pipeline (MLX)	55 / 3,470
07-14	`2d11adf`	v1.0-evaluation	End-to-end VLM evaluation framework	45 / 4,419
07-14	`ba457c4`	—	Evaluation follow-up (tiny fix)	3 / 4
07-14	`638b7bd`	—	Stabilize evaluation deps + CI	6 / 42
07-14	`3774492`	v0.1.0-ci-green	Self-contained evaluation fixtures	1 / 44
07-14	`380b18e`	—	Mango training pipeline (config-only)	6 / 101
07-14	`673ed77`	v0.2.0	Generalize evaluation for multiple crops	19 / 501
07-17	`088c134`	—	Streamlit inference/demo app	21 / 2,492
07-17	`66d428e`	v1.0.0	README overhaul (public entry point)	8 / 197
07-17	`9f08d99`	—	Presentation polish (figures, Mermaid)	11 / 370

EVIDENCE

`git log --reverse --date=short`, `git tag -l --sort=creatordate`, and per-commit `git show --stat`. Repository creation time from the GitHub API (`createdAt: 2026-07-11T12:12:19Z`).

2.2 Day by day, with the "why"

Day 1 — 2026-07-11 · Foundations

The first commit was deliberately *scaffold-only*: the entire package layout, every subpackage that exists today (all stubbed with typed interfaces raising `NotImplementedError`), `pyproject.toml` (hatchling, ruff, mypy-strict, pytest), the config system, the test scaffold, developer docs, and the full `plantdx` CLI surface

— **zero pipeline logic**. Later the same day, the first real functionality landed: the Dataset Audit and Normalization engines. Auditing the raw data immediately produced the project's first surprise (§6): the "tomato" download was the *entire* 38-class PlantVillage dataset, which is *why* a Normalization engine had to exist at all.

> **LESSON EMBEDDED IN DAY 1**
> "Make it look like a mature open-source project before adding functionality." Scaffolding the full typed surface first meant every later milestone slotted into a fixed contract instead of reshaping the package. The cost — over-scaffolding — was paid down deliberately on Day 2 (§13).

Day 2 — 2026-07-12 · The knowledge layer

Three commits: the deterministic **ontology compiler** (DKB → typed knowledge graph, content- hashed, fail-closed validation), a dedicated **stabilization** pass (which, notably, *deleted* an unused Protocol layer and consolidated over-scaffolded code — the first refactor), and the **vocabulary + symptom lexicon** compiler. The stabilization commit is the largest by files touched (86) because it also finalized developer documentation.

Day 3 — 2026-07-13 · The language layer and first training

The caption pipeline landed in one large commit (concept models → templates → planner → generator → validator → corpus → exporters), tagged twice (`v0.6-generation` and, once the corpus content-hash was pinned, `v0.6-corpus-freeze`). The same day, the tomato QLoRA training pipeline was completed. The jump from `v0.6` to `v0.8` (skipping v0.7) is a real artifact of milestone-numbering, not a lost release.

Day 4 — 2026-07-14 · Evaluation, then multi-crop

The busiest day: the end-to-end evaluation framework (`v1.0-evaluation`), a small follow-up, a dependency/CI stabilization (`638b7bd` — the numba/NumPy story, §11), a fixture self-containment fix that finally got CI genuinely green (`v0.1.0-ci-green` — note the switch to semver), the mango training *config* (six files — mango needed almost no new code), and the **evaluation generalization** that made crop a derived value rather than a hardcoded constant (`v0.2.0`). This last change is the project's most instructive bug-and-fix (§10, §12).

The gap, then Day 7 — 2026-07-17 · The product and the polish

After a three-day gap, three commits delivered the Streamlit demo (real confidence, adapter verification, organized uploads, logging), a README overhaul (`v1.0.0`), and a final presentation pass (Mermaid diagrams, re-rendered figures). These are the only commits with detailed bodies, so their rationale is captured directly in git.

> **ENGINEERING NOTE — THE TWO "STABILIZATION" COMMITS**
> Two separate commits exist solely to make the tree green without changing behavior: `584f49e` (style/CI + docs) and `638b7bd` (evaluation deps + CI), plus `3774492` (test self-containment). Roughly **19%** of all commits are pure stabilization. That ratio is itself a finding: on a fast, single-author build, dedicated "make it green, change nothing" passes were necessary and worthwhile (§11, §13, §15).

3 · Research Evolution

This is the intellectual spine of the project. Each research decision below is presented with the reasoning that produced it and the evidence that records it. The through-line: *constrain the system until the property you want ("grounded, reproducible captions") is impossible to violate.*

3.1 Why captioning, not classification

A pure classifier would output a label; PlantDx instead fine-tunes a VLM to **describe** the leaf in grounded natural language, from which a disease id is *extracted*. Two reasons: (1) the research contribution is a method for building *caption* supervision that is scientifically grounded — a classifier has nothing to ground; (2) a caption carries the observable evidence (colors, shapes, distribution) that justifies the label, which is exactly what a knowledge-grounded system can verify. Classification metrics are still computed, but by parsing the generated caption, not by a classification head.

EVIDENCE

The generation pipeline (`concepts → templates → corpus`) produces captions, not labels; classification is a downstream extraction step (`evaluation/classification.py`: `extract_disease_id`). The "describe, don't classify" instruction is the training prompt: `assets/training/instructions.json` ("Describe the visible condition of this … leaf.").

3.2 Why VLMs, and why these VLMs

The task is inherently multimodal (image → grounded text), so a vision–language model is the natural substrate. Four **declared target models** were chosen to represent the open-weight VLM landscape; only one was trained in this window (§8). The choice of `Qwen2.5-VL-7B-Instruct-4bit` as the first model was pragmatic: a strong instruction-tuned VLM available as a 4-bit MLX build, small enough to fine-tune on 24 GB.

EVIDENCE

`knowledge_base/dkb.json → metadata.target_models = [Qwen3-VL-8B-Instruct, Qwen2.5-VL-7B-Instruct, InternVL3-8B, Gemma-3-12B]`. Trained model: `configs/models/qwen25vl.yaml → model_path: mlx-community/Qwen2.5-VL-7B-Instruct-4bit`.

3.3 Why parameter-efficient fine-tuning → LoRA → QLoRA

Full fine-tuning of a 7B model does not fit in 24 GB of unified memory. LoRA trains small low-rank adapters over frozen weights. Because the base model is *already* 4-bit quantized, "LoRA over a quantized base" **is** QLoRA by construction — it was not a separate design step, it fell out of the model choice. This is the single most consequential efficiency decision: it made local training feasible at all.

RESEARCH NOTE — QLORA "BY CONSTRUCTION"

The LoRA config states it explicitly: "the base model is already 4-bit MLX, so LoRA-over-quantized- base is QLoRA by construction. Lowest memory footprint." The trained adapter is **40,370,176** parameters (rank 16) — ~0.5% of the 7B base.

EVIDENCE

`configs/lora/qlora.yaml` (method `qlora`, rank 16, alpha 32, dropout 0.05); the validator requires a 4-bit base for qlora (`training/config.py`: "qlora requires a 4-bit quantized model"); adapter parameter count from `checkpoints/qwen25vl_tomato_qlora/adapters.safetensors`.

3.4 Why MLX, Apple Silicon, and local inference

The development machine is an Apple M4 Pro (24 GB unified memory). MLX is Apple's array framework and `mlx-vlm` its VLM training/inference layer; together they make 7B QLoRA fine-tuning and inference run on the integrated GPU without a discrete accelerator or cloud. Local inference also serves the research thesis: a reproducible, self-contained artifact anyone with the same hardware can rerun — no API keys, no external service, no data leaving the machine.

EVIDENCE

`docs/TRAINING.md` ("Apple Silicon M4 Pro, 24 GB unified memory, MLX / mlx-vlm / Metal"); `pyproject.toml` `[train]` extra (mlx, mlx-vlm); the training command builds `python -m mlx_vlm.lora …` (`training/command.py`).

3.5 Why benchmark first — the finding that shaped everything

The project's first research act was to ask whether an off-the-shelf VLM could do the job zero-shot. It could not (§1.3: 7.3% / 17.5%). That single fact is the reason models are banned from the caption path: if you cannot trust a VLM to *diagnose* zero-shot, you cannot trust it to *describe*, so it must never author or judge training captions.

3.6 Why dataset verification became necessary

The audit engine was not planned as a research contribution — it became necessary the moment the first audit ran and revealed the "tomato" folder was the *entire* multi-crop PlantVillage dataset (38 classes, ~54k images), not a filtered subset. You cannot ground captions in a dataset you have not verified; the Normalization engine exists **because** of what the audit found (§6).

3.7 Why an ontology became necessary

The DKB is prose-heavy per-disease records. Without an intermediate typed layer, every downstream stage (vocabulary, concepts, generation) would re-parse that prose and re-implement the observability rule and severity rule independently — duplicated logic that silently drifts. The ontology re-expresses the DKB as a typed, evidence-linked graph so "is this symptom observable?" and "is this claim licensed?" become graph queries with *one* correct implementation.

EVIDENCE

`ontology_design/01_architecture.md` (why the ontology exists; the explicit rejection of both "consume DKB prose directly" and "full OWL/RDF + reasoner"). Realized graph: 574 concepts, 1,289 edges, content-hash `sha256:25ae0f6d…` (`tests/unit/ontology/test_domain_determinism.py`).

3.8 Why deterministic caption generation

A scientific artifact that cannot be independently rebuilt is a claim, not an artifact. So every stage is a pure function of declared inputs, every output carries a content hash computed only from semantic content (never timestamps or paths), and rebuilding must be byte-identical. Determinism is enforced by golden-hash regression tests, not assumed.

> **EVIDENCE — THE FOUR FROZEN CONTENT HASHES**
>
> ontology `sha256:25ae0f6d…`, vocabulary `sha256:99d5833c…`, concepts `sha256:e6f4870b…`, corpus `sha256:1225eb86…` — each pinned in a `test_*_determinism.py` and treated as "sacred: any change is a reviewed version event, never a silent side effect."

3.9 Why "expert" (cited) captions and no hallucination by construction

Every fact a caption may assert must exist in the DKB, which is hand-curated and cited against APS, UC IPM, UF/IFAS, CABI, and peer-reviewed literature. Hallucination is not "avoided by prompting" — it is made *structurally impossible*: a caption can only assert a symptom that (a) is a node in the ontology, (b) is marked observable, and (c) carries a resolvable evidence reference. There is no code path by which an unlicensed claim enters a caption.

> **EVIDENCE**
>
> `knowledge_base/dkb.json → metadata.reference_registry` (citation keys → sources); the ontology validator's evidence-closure check (`V-ONT-7`) and observability checks (`V-ONT-8`); the caption validator battery `V-CAP-1..12`.

3.10 Why evaluation changed over time

Evaluation began as a single-model, tomato-only comparison (base vs fine-tuned). When mango arrived, a real bug surfaced: the evaluation had hardcoded "tomato" in three independent places, so evaluating mango silently used tomato's label map, lexicon, and output directory. The fix generalized evaluation to **derive crop from the dataset manifest** — never hardcode it. This is the clearest example of research infrastructure evolving under the pressure of a second data point (§10, §12).

> **EVIDENCE**
>
> Commit `673ed77` "generalize evaluation pipeline for multiple crops" (19 files, +501/–93). The derived-crop mechanism: `evaluation/config.py → resolve_crop()` reading `<dataset>/manifest.json → crop`.

4 · Repository Evolution

The repository evolved as a sequence of tagged versions, each adding one compiler or capability on top of a fixed, typed scaffold. Because the full package surface was stubbed on Day 1, "evolution" here means *filling in* contracts and, occasionally, deleting scaffolding that proved unnecessary.

4.1 Versions, mapped to tags

Version	What became real	What was still a stub
v0.1-scaffold	Package layout, config system, CLI surface, tests, tooling, docs — all typed, all stubbed.	Everything (by design).
v0.3-normalization	Audit + Normalization engines; real config loader, io/hashing/logging utils.	Ontology, vocabulary, generation, training, evaluation.
v0.4-ontology	Domain ontology compiler (`ontology/domain/`).	Vocabulary, language layer, training, evaluation.
v0.5-stabilization	CI/style green; dev docs finalized; first refactor (deleted a Protocol layer, consolidated converters).	Vocabulary and downstream.
v0.5-vocabulary	Vocabulary + symptom lexicon compiler (`vocabulary/domain/`).	Concepts, templates, corpus, training, evaluation.
v0.6-generation / -corpus-freeze	Concept models, template engine, planner/generator/validator, corpus, exporters.	Training, evaluation, image grounding.
v0.8-tomato-training	Tomato QLoRA training workflow (`training/` + `training/data/`).	Evaluation; other crops.
v1.0-evaluation	Two-stage evaluation framework (`evaluation/`).	Multi-crop; the demo.
v0.1.0-ci-green → v0.2.0	Self-contained eval fixtures; mango training config; crop-agnostic evaluation.	Image-grounded builder; other 3 models.
v1.0.0 (+ polish)	Streamlit demo app; README as public entry point; Mermaid + real figures.	Image grounding; Qwen3-VL/InternVL3/Gemma-3.

THE VERSIONING ANOMALY, EXPLAINED

The tags jump `v0.1 → v0.3` (no v0.2) and `v0.6 → v0.8` (no v0.7), then switch scheme entirely to semver at `v0.1.0-ci-green`. The gaps are milestone-numbering choices (audit + normalization were tagged together as v0.3; training as v0.8), not lost work. The semver switch marks the moment the project started thinking in *releases* rather than internal milestones — right after CI first went genuinely green.

4.2 Modules added over time

```
src/plantdx/
  core/ config/ utils/        added v0.1 (scaffold) — shared types, config, io/hashing/logging
  audit/ normalization/       added v0.3 — dataset inventory + canonicalization
  ontology/domain/            added v0.4 — DKB → typed graph compiler
  vocabulary/domain/          added v0.5 — graph → controlled vocabulary + lexicon
  concepts/ templates/        added v0.6 — per-disease concept models + authored templates
  corpus/ exporters/          added v0.6 — planner/generator/validator + format exporters
  training/ training/data/    added v0.8 — mlx-vlm QLoRA workflow + image×caption builder
```

```
  evaluation/                 added v1.0 — two-stage base-vs-fine-tuned framework
app/                          added at public-release (07-17) — Streamlit demo layer
```

4.3 Modules deliberately kept as stubs (not deleted)

A distinctive property of the repo: several M1-era stub packages coexist with the real implementations that superseded them, rather than being deleted. This is intentional — they mark the *frozen design surface* for future milestones (image grounding, the other three models).

Stub	Status	Superseded / awaited by
`ontology/{builder,models,concept_schema}`	stub (M1)	the real `concepts/` package
`vocabulary/{builder,lexicon,expander}`	stub (M1)	the real `vocabulary/domain/`
`generation/ validation/ diversity/`	stub	M4 image-grounded pipeline
`dataset/ (emitter, splits, converters)`	stub	M4 Instruction Dataset Builder
`knowledge_base/loader.py (DKBLoader)`	stub, dead	the ontology compiler's own loader
`evaluation/{zero_shot,compare}.py`	stub	the four-model zero-shot matrix (descoped)

LESSON — "NEW PACKAGE SUPERSEDES OLD STUB"

Twice (ontology, vocabulary) a fresh `domain/` subpackage was written next to an M1 stub rather than editing the stub, because the stub's designed dependency (the caption-concept model) did not exist yet. Re-founding onto the real substrate (the compiled ontology) was cleaner than forcing the old contract. The cost: two "ontology"/"vocabulary" names in the tree — a documented naming collision (`CLAUDE.md §4.2–4.3`).

4.4 Deletions and consolidations

The repository is unusual in how *little* it deleted — but the deletions it did make were deliberate simplifications, all in the v0.5 stabilization pass:

- Removed `core/interfaces.py` — an unused Protocol layer duplicating concrete ABCs.
- Converted `SeedDeriver` (class) → pure functions (`image_seed` , `caption_seed` , `attempt_seed`).
- Consolidated a 7-file `dataset/converters/` package into one `dataset/converters.py` .
- Renamed `dataset/schema.py → serialization.py` (to disambiguate from `config/schema.py`).
- At public release: removed two superseded figure PNGs, tidied `.DS_Store` .

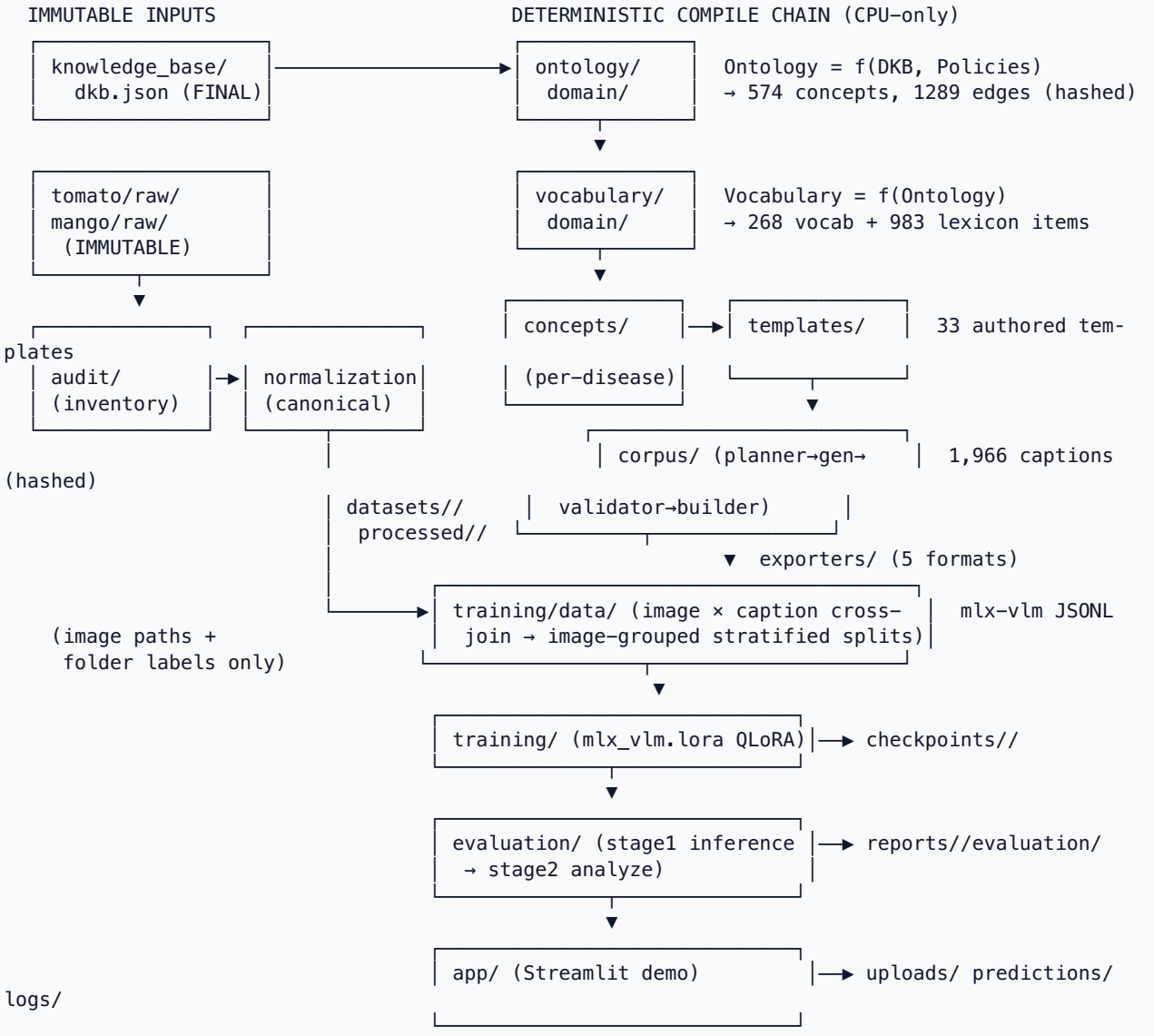
EVIDENCE

`CHANGELOG.md` "post-review simplification" entry; commit `584f49e` (`chore: stabilize repository` , 86 files, +1,611/–567 — the negative delta is the deletions).

5 · Complete Architecture

PlantDx is a chain of deterministic compilers plus two runtime pipelines (training, evaluation) and a demo. The governing law is written in the project's own memory: *Ontology = f(DKB, Policies); Vocabulary = f(Ontology); Corpus = g(Ontology, Vocabulary, Templates, seed).* Every stage is a pure function of declared inputs, sorted before serialization, and content-hashed.

5.1 End-to-end data flow



5.2 Folder-by-folder

Path	Responsibility	Key property
`knowledge_base/`	The DKB (18 diseases × 46 fields, cited) + human-readable report.	FINAL, single source of truth.
`caption_framework/`	8-doc frozen spec for the caption pipeline (D–I components).	Design only, no code.

`ontology_design/`	13-doc frozen spec for the domain ontology.	Design only, no code.
`src/plantdx/core/`	Enums, exceptions, dataclasses, seed fan-out.	Leaf package, no internal deps.
`src/plantdx/config/`	Pydantic schema + YAML loader (`configs/*.yaml → PlantDxConfig`).	Composed via `includes:` .
`src/plantdx/audit/`	Image inventory, corrupt/duplicate detection, checksums.	Read-only; never decodes fully.
`src/plantdx/normalization/`	Extract relevant classes → `datasets/<crop>/processed/` .	Raw is immutable; SHA-256 verified.
`src/plantdx/ontology/domain/`	DKB → typed knowledge graph (8 files).	Fail-closed `V-ONT-1..11` .
`src/plantdx/vocabulary/domain/`	Ontology → controlled vocabulary + bounded lexicon.	Fail-closed `V-VOC-1..9` .
`src/plantdx/concepts/`	Per-disease mandatory/optional/forbidden concept sets.	Fail-closed `V-CON-1..11` .
`src/plantdx/templates/`	33 authored templates + compatibility routing.	Fail-closed `V-TPL-1..8` .
`src/plantdx/corpus/`	Planner → generator → validator → corpus builder.	Fail-closed `V-CAP-1..12` .
`src/plantdx/exporters/`	Reshape corpus → generic/llava/paligemma/blip2/messages.	Byte-identical reshapers.
`src/plantdx/training/`	Config, data builder, command builder, planner, checkpoints, runner, inference.	Orchestrates `mlx_vlm.lora` ; lazy MLX.
`src/plantdx/evaluation/`	Two-stage inference/analyze, metrics, hallucination, clinical, stats, figures.	Crop derived from manifest.
`app/`	Streamlit demo (inference reuse, confidence, adapter panel, eval panel).	Presentation only; no production change.
`configs/`	Master + paths + per-stage + per-crop training YAML.	Composed, validated, hashed.
`assets/`	Authored inputs (templates, label map, instruction banks).	Version-controlled inputs.
`tests/`	unit / integration / benchmark, mirroring `src/` + `app/` .	Golden-hash + fail-mode coverage.

5.3 The training pipeline (control flow)

```
plantdx train --config configs/train/qwen25vl_<crop>.yaml
   │
   ├─ load_training_config()   compose run × model × lora YAML → typed TrainingConfig (fail-closed)
   ├─ build_training_dataset() label_map → discover images → load caption pool → pair →
   │                           image-grouped stratified split → write train/val/test.jsonl + manifest.json
   ├─ build_plan()             iters, effective batch, time/mem/disk estimates, LR-schedule preview
   ├─ resolve_layout()         checkpoints/ (best/latest/resume)
   ├─ build_command()          exact `python -m mlx_vlm.lora …` argv
   ├─ build_report()           training_plan.md + .json (ends in the one launch command)
```

```
   └─ launch()  [only without --dry-run]  subprocess mlx_vlm.lora → checkpoints//ad-
apters.safetensors
```

EVIDENCE

`src/plantdx/training/runner.py` (`prepare_run`, `launch`); `training/command.py`; `docs/TRAINING.md`. The dataset builder reads **paths and folder labels only** — never pixels, never an LLM/VLM.

5.4 The evaluation pipeline (two stages, one contract)

```
plantdx evaluate --stage all --adapter checkpoints/ --dataset artifacts/training//dataset
   │
   ├─ STAGE 1  inference  (needs mlx-vlm)     resolve_crop(manifest) → base + fine-tuned generate
   │                                          → predictions.jsonl + metadata.json  (the ONLY con-
tract)
   └─ STAGE 2  analyze    (needs [eval] stack) reads predictions.jsonl ONLY →
        classification (sklearn) · text metrics (BLEU/ROUGE/METEOR/CIDEr/BERTScore) ·
        per-disease · hallucination · clinical · latency · paired stats · figures · manifest
        → reports//evaluation/*
```

WHY TWO STAGES THAT NEVER SHARE MEMORY

Inference needs **mlx-vlm** (Apple Silicon); the metrics stack needs **matplotlib/sklearn/scipy/nltk/pycocoevalcap/rouge-score/bert-score+torch** — two heavy dependency sets that are never installed together. Communicating only through a frozen `predictions.jsonl` lets each stage run in its own environment, on different machines if needed. This is a deliberate, load-bearing architectural choice (`evaluation/runner.py`, `docs/EVALUATION.md`).

5.5 The inference/demo pipeline

```
streamlit_app.py → app/ui.py
   upload → validate (PIL) → temp file → app/inference.run_inference()
      ├─ get_model_handle()   st.cache_resource (load ONCE per crop; max_entries=1)
      ├─ stream_generate()    accumulate per-token selected-token probability = REAL confidence
      ├─ classify()           reuse frozen extractor + normalized/alias fallback (crop-scoped)
      ├─ _status()            confident / low_confidence / unknown (vs threshold)
      └─ _release_mlx_cache() bound + clear MLX buffer cache (stable memory)
   → save original to uploads/// + JSON sidecar
   → predictions/.json + history.json + logs/predictions.jsonl
```

5.6 Configuration, logging, visualization

- **Configuration:** `configs/config.yaml` composes the others via `includes:`; the loader validates against a Pydantic schema and computes a `config_hash`. Training uses a three-layer composition (run × model × lora).
- **Logging:** the pipeline stages write deterministic, timestamp-free build logs; the demo adds a Python file logger (`logs/plantdx_app.log`) plus an append-only JSONL prediction log.
- **Visualization:** evaluation figures come from `evaluation/visualize.py` (a CVD-validated palette, PNG+SVG, metrics never mixed on one axis); the README figures are re-rendered by `scripts/render_readme_figures.py` from the same CSVs.

EVIDENCE

`configs/config.yaml`, `config/loader.py`; `app/logging_setup.py`, `app/storage.py`; `src/plant-dx/evaluation/visualize.py`, `scripts/render_readme_figures.py`.

6 · Dataset Story

PlantDx uses two public datasets, treated as **immutable ground truth** — never renamed, moved, or edited. Everything downstream consumes a *normalized* copy, never the raw folders. The dataset story is short but consequential: one audit finding reshaped the pipeline.

6.1 Sources

Crop	Dataset	Raw layout	License / citation
Tomato	**PlantVillage** (full, multi-crop)	`tomato/raw/PlantVillage/{train,val}/<ClassFolder>/`	Hughes & Salathé, 2015 (arXiv:1511.08060). Not redistributed.
Mango	**MangoLeafBD**	`mango/raw/MangoLeafBD/<ClassFolder>/` (flat)	Ahmed et al., 2023, *Data in Brief* 47:108941, CC BY 4.0. Not redistributed.

6.2 The audit finding that created a whole engine

RESEARCH NOTE — THE "TOMATO" FOLDER WAS NOT TOMATO

The first audit revealed the `tomato/raw/PlantVillage/` download was the **complete multi-crop PlantVillage dataset** — a `train/` + `val/` split, **38 classes** spanning many crops (tomato, potato, corn, grape, apple, …), **~54,305 images** — not a pre-filtered 10-class tomato subset as originally assumed. This single discovery is *why* the Normalization engine exists: you must extract and canonicalize the relevant classes before anything downstream can trust the data.

EVIDENCE

A real `plantdx audit` run: **0 corrupt images** across both datasets (58,305 total inspected), 38 raw tomato classes discovered vs 10 expected, 21 exact-duplicate groups per dataset. Recorded in `CLAUDE.md §8.3`, `docs/AUDIT.md`, and the audit's own `reports/` output.

6.3 Cleaning, verification, normalization

- **Corrupt handling:** metadata-only inspection via Pillow (never a full decode); one bad image can never crash the run. Result: 0 corrupt across 58,305 images.
- **Duplicates:** exact-duplicate detection via SHA-256 always runs (21 exact-duplicate groups each — coincidentally the same count for both datasets, unrelated). Perceptual near-duplicate detection exists but is disabled by default (it requires decoding pixels).
- **Normalization:** extracts only the relevant classes, applies a canonical class map (`Tomato___Early_blight → early_blight`), merges any train/val split into unified class directories (recording each image's original split in the manifest), and verifies every copied file's SHA-256 against its source.

6.4 The normalized datasets, with real counts

Crop	Classes	Normalized images	Train / Val / Test
Tomato	10	18,006	16,201 / 895 / 910

Mango	8	4,000	3,600 / 200 / 200

EVIDENCE

`artifacts/training/qwen25vl_<crop>_qlora/dataset/manifest.json` (`image_count`, `rows_per_split`, `classes`). Note: these authoritative manifest counts (tomato = 18,006) supersede an earlier "~14k" estimate in `CLAUDE.md §8.1` — where the memory doc and the artifact disagree, the artifact wins.

6.5 Label inconsistencies, imbalance, and the one coupling

- **Folder-name coupling:** the only place a filesystem name meets a knowledge id is `assets/metadata/label_map.json` (`<class folder> → disease_id`), kept tiny and explicit so a domain expert can verify it by reading one file.
- **Class imbalance (tomato):** real and significant — Yellow Leaf Curl Virus dominates the test split (269 images) while Mosaic Virus is rare (20). This shows up in evaluation: rare classes have noisier per-class F1 (§10).
- **Class balance (mango):** perfectly balanced — 500 images/class, exactly 25 per class in the test split.
- **Naming variance across mirrors:** the frozen spec warns PlantVillage folder spelling varies by mirror (`caption_framework/04_dataset_schema_spec.md` §2); the label map is reconciled against the actual normalized folder names.

6.6 What was NOT found

EVIDENCE LIMIT

No evidence of formally rejected datasets was found in the repository or history — PlantVillage (tomato subset) and MangoLeafBD are the only datasets referenced. **No missing-label incidents** are recorded beyond the folder-name-mapping design. The 28 non-tomato PlantVillage classes are *ignored, not deleted* — left untouched in `raw/` and recorded by name in `datasets/tomato/class_mapping.json:ignored_folders`.

LESSON — AUDIT BEFORE YOU ARCHITECT

The most expensive assumption in the project ("the tomato folder contains tomatoes") was falsified in minutes by an automated audit. Verifying data *first* turned a latent, silent failure into a one-line finding and a well-scoped engine.

7 · Caption Evolution

Captions are not "prompted from a model" — they are *compiled*. So "caption evolution" here is the story of a generator getting more grammatical, more diverse, and more honest, verified against the *real* DKB rather than fixtures. The pivotal episode is the RC1 hardening pass.

7.1 How a caption is built

```
DKB fields ➔ Concept Model (per disease: mandatory/optional/forbidden concepts,
                            order, information budget, register, controlled realizations)
           ➔ Template (structured segments: lit / slot / opt / list, with glue/suffix)
           ➔ Sentence Planner (seed-chosen controlled realizations → SentencePlan)
           ➔ Generator (realize to English + deterministic surface repair)
           ➔ Validator (V-CAP-1..12, independent of the generator)
           ➔ Corpus Builder (enumerate, dedup, drop-and-record failures)
```

WHY A STRUCTURED SEGMENT SCHEMA, NOT INLINE-STRING PATTERNS

Templates use a structured schema (`lit` / `slot` / `opt` / `list` with `glue` / `suffix`) rather than the spec's inline-string patterns, so deleting an optional slot is grammatical *by construction* — no fragile regex repair. This was a deliberate deviation from the spec, documented as such.

7.2 The instruction prompts

The training-time instruction is a pure task prompt carrying no disease knowledge and asserting no symptom — the response always comes verbatim from the frozen corpus. When mango arrived, a crop-worded variant was added rather than reusing tomato's wording (a real correctness fix, §12).

EVIDENCE

`assets/training/instructions.json` ("Describe the visible condition of this tomato leaf.", +7 paraphrases) and `assets/training/instructions_mango.json` (mango wording).

7.3 RC1 — the caption-hardening pass (the real "prompt engineering")

Before freezing the corpus, a publication-grade audit ran the generator against the *real* 18-condition DKB and caught grammar/quality bugs that synthetic fixtures had missed. Three were genuine failures of the generator on real phrasing:

Bug (real DKB text)	Symptom	Fix
Clause phrases as `{primary}` ("begin on oldest leaves")	ungrammatical "X are visible"	deterministic noun-phrase filter on primary realizations
Number agreement ("a coating are visible", "spider mites is present")	singular/plural mismatch	make the few number-fragile templates number-neutral ("can be seen")
Negation leak ("none characteristic")	a negation phrase used as a realization	first-word negation filter

LESSON — REAL DATA BEATS FIXTURES

All three bugs were invisible to the synthetic test fixtures and only appeared when the generator ran against the actual DKB. The fixtures were *then* extended to reproduce each case. "Verify against the real inputs, not a mock" is a recurring theme (§11, §15).

7.4 Diversity, the healthy class, and severity honesty

- **Diversity (W6):** bounded per-(template, subset) realization variants drawing on each disease's multiple DKB phrasings grew the corpus from **1,070** → **1,966** captions; per-disease minimum **6** → **15**; unique bi-grams/trigrams up ~12%, all still unique and validated.
- **Healthy class (W5):** `healthy_state` was decomposed into multiple atomic, evidence-supported observations from the DKB healthy fields — healthy captions **12** → **32** — plus 3 new healthy templates.
- **Severity honesty:** the source data has no per-image severity label, so no caption may assert one. Severity-stage tokens (`STAGE_TOKENS`) are filtered from realizations at the source, so the `V-CAP-11` rejection count dropped **17** → **0** (validators retained as defense in depth). DKB phrases containing "severe" are rejected by two validators and dropped.

EVIDENCE

`CHANGELOG.md` "RC1: caption corpus hardening" and `CLAUDE.md §5`; final corpus content-hash `sha256:1225eb86…` (1,966 captions), pinned in `tests/unit/corpus/test_corpus_determinism.py`.

7.5 "Expert review" — what it actually is

EVIDENCE LIMIT

The "expert" in PlantDx is the **cited DKB** itself — hand-curated against APS / UC IPM / UF IFAS / CABI / peer-reviewed sources, with a `reference_registry` resolving every claim to a source. **No separate human-in-the-loop caption review process** (annotators, adjudication rounds) is present; the `qa/` package that would host sampling/review/acceptance is an unimplemented stub. Caption trustworthiness is therefore *structural* (grounding + validators), not the product of a documented manual review pass.

8 · Model Evolution

This section requires the most discipline, because the user's brief lists models to *consider* that do not all appear in the repository. The rule here is absolute: **a model is documented only if it appears in the code, configs, or history; and measured properties are stated only where a real measurement exists.**

8.1 The one model actually trained and evaluated

Property	Qwen2.5-VL-7B-Instruct (4-bit MLX)
Role	Base + fine-tuned (the only trained model in this window)
Tomato accuracy (base → fine-tuned)	7.3% → 93.7%
Mango accuracy (base → fine-tuned)	17.5% → 82.0%
Trainable LoRA params	40,370,176 (rank 16, scale 2.0)
Load time (measured)	~3 s (once per crop, cached)
Inference latency (measured)	~0.8–1.7 s / image (warm, greedy)
Active memory (measured)	~5.8 GB, flat across many predictions

EVIDENCE

`configs/models/qwen25vl.yaml` (4-bit, `max_seq_length 2048`, `image_resize 448`); `checkpoints/qwen25vl_<crop>_qlora/adapter_config.json` (rank 16, scale 2.0, 196 target keys × 28 layers); `reports/<run>/evaluation/metrics.json`; latency/memory measured directly via `mlx.core.get_active_memory()` during a run.

8.2 Declared target models — scaffolded, not trained

Three additional models are *declared* as targets and have *scaffolded converter classes*, but were **not trained or evaluated** in this window. There are no measured accuracy, memory, speed, or hallucination numbers for them — and this document will not invent any.

Model	Where it appears	Status
Qwen3-VL-8B-Instruct	DKB `target_models`; `Qwen3VLConverter` stub	Declared + scaffolded; not trained. No measurements.
InternVL3-8B	DKB `target_models`; `InternVL3Converter` stub	Declared + scaffolded; not trained. No measurements.
Gemma-3-12B	DKB `target_models`; `Gemma3Converter` stub	Declared + scaffolded; not trained. No measurements.

EVIDENCE

`knowledge_base/dkb.json → metadata.target_models`; `src/plantdx/dataset/converters.py` (`CONVERTER_REGISTRY` = Qwen2.5-VL, Qwen3-VL, InternVL3, Gemma-3, generic MLX — bodies raise `NotImplementedError` until M4); `core.enums.TargetModel`.

8.3 "Models" that are actually export formats, not fine-tuned models

A NAMING TRAP, DISAMBIGUATED

The strings `llava`, `paligemma`, and `blip2` appear in the repository — but as **dataset export formats** (serialization shapes for the caption corpus), not as VLMs being fine-tuned. `src/plantdx/exporters/formats.py` defines `_generic`, `_llava`, `_paligemma`, `_blip2`, `_messages`. Treating them as trained models would be a factual error.

8.4 Models in the brief that are NOT in the repository

NOT ENOUGH EVIDENCE — STATED EXPLICITLY

The brief lists **Molmo, Pixtral, and Florence**. A full-repository search (`.py`, `.md`, `.json`, `.yaml`) returns **zero occurrences** of any of them. There is **no evidence** that Molmo, Pixtral, or Florence were ever evaluated, referenced, or considered in this repository. They are omitted rather than fabricated.

8.5 Why the model story looks like this

The architecture was deliberately built model-agnostic (one canonical caption library, per-model converters behind a registry) precisely so that adding the other three declared models is a converter-implementation task, not a redesign — but that task belongs to a future milestone (M4/M5). The single-model result (Qwen2.5-VL) is the proof-of-method; the multi-model comparison is scaffolded and awaited.

9 · Training Evolution

Training is a **config-driven orchestration** of `mlx-vlm`'s tested LoRA trainer, not a hand-rolled MLX loop. The design goal: a training run is a pure function of one YAML file (plus the frozen corpus and normalized images). Adding mango required almost no new code — six files, mostly a config.

9.1 The three-layer config

A run composes a **run** file × a **model** file × a **LoRA method** file:

```
configs/train/qwen25vl_<crop>.yaml   ── the run (seed, optimizer, data, checkpoints, logging)
   ├─ model: qwen25vl  ──────────────► configs/models/qwen25vl.yaml
   └─ lora:  qlora     ──────────────► configs/lora/qlora.yaml
```

9.2 The actual hyperparameters (both crops, identical)

Hyperparameter	Value	Hyperparameter	Value
Learning rate	1.0e-4	Min LR	1.0e-6
Scheduler	cosine	Warmup steps	100
Optimizer batch size	1	Grad accumulation	8 (eff. batch 8)
Epochs	1	Grad clip	1.0
Weight decay	0.0	Train on completions	true (loss on caption only)
LoRA rank	16	LoRA alpha	32
LoRA dropout	0.05	Target modules	q/k/v/o + mlp (lang model)
Max seq length	2048	Image resize	448×448
Quantization	4-bit	Grad checkpointing	true (memory)
Seed	20260711	Train vision tower	false (LM only)

EVIDENCE

`configs/train/qwen25vl_tomato.yaml`, `configs/train/qwen25vl_mango.yaml`, `configs/models/qwen25vl.yaml`, `configs/lora/qlora.yaml`. The command builder emits the exact `python -m mlx_vlm.lora … --lora-rank 16 --lora-alpha 32 …` argv.

9.3 Scale of the two runs

Run	Rows	Iterations (1 epoch)	Est. wall-clock
Tomato	16,201 train	16,201	~405–945 min
Mango	3,600 train	3,600	~90–210 min

Grad accumulation of 8 with a micro-batch of 1 keeps peak memory ~8–14 GB of 24 GB while giving an effective batch of 8 — the central memory/throughput trade at this scale. Checkpoints save every 200 steps (keep best + last 3, resume supported).

9.4 What the completions objective means

TRAIN_ON_COMPLETIONS = TRUE

The prompt tokens are masked; loss is computed on the caption (the "answer") only. This teaches the model to *produce* grounded descriptions rather than to reconstruct the instruction — appropriate for instruction-tuning where the instruction is fixed task framing.

9.5 Failures, guards, and restarts

- **DoRA guard:** LoRA/QLoRA/DoRA are config-selectable, but the installed backend (mlx-vlm 0.6.x) cannot run DoRA — so `plantdx train` **fails closed** with a clear message rather than silently running LoRA. (`training/lora.py`: `check_method_supported`.)
- **QLoRA precondition:** the config validator rejects `qlora` unless the model is 4-bit — a fail-closed check that prevents a mis-specified run.
- **Backend/version sensitivity:** training runs against a specific `mlx-vlm` build; the version matters for kernel support (see §11 for the M-RoPE VJP episode encountered during real QLoRA training and resolved by the `mlx-vlm 0.6.4` upgrade).

EVIDENCE — TRAINING ACTUALLY COMPLETED

Real adapters exist for both crops: `checkpoints/qwen25vl_<crop>_qlora/adapters.safetensors` (40,370,176 params) plus periodic snapshots (e.g. mango's `0000200…0003600_adapters.safetensors`). The evaluation numbers in §10 are downstream proof the training succeeded.

9.6 A note on in-loop evaluation

HONEST LIMITATION

mlx-vlm 0.6.x trains on the `train` split and passes `val_dataset=None` in its CLI path, so `--steps-per-eval` / `--val-batches` do *not* evaluate a held-out set *during* training. The `validation.jsonl` / `test.jsonl` splits are written and reserved for the separate evaluation stage. There is therefore no in-training loss/val curve artifact; evaluation is entirely post-hoc (§10).

10 · Evaluation Evolution

Evaluation is the project's most-evolved runtime component: it went from a single-model, tomato-only comparison to a crop-agnostic framework — and the transition exposed the single most instructive bug in the repository (§12). It is deliberately rigorous: official reference metric implementations, not approximations.

10.1 What is measured, and with what

Family	Metrics	Implementation
Classification	accuracy, balanced acc, macro/weighted/micro P/R/F1, confusion matrix, unclassified count	scikit-learn (reference)
Caption quality	BLEU-1..4, CIDEr	`pycocoevalcap` (MS-COCO convention; CIDEr over the full batch)
	ROUGE-L	Google `rouge-score`
	METEOR	`nltk` + WordNet
	BERTScore-F1	`bert-score` (roberta-large)
Grounding	hallucination (other-disease/pathogen/treatment/crop/impossible-symptom), clinical correctness	deterministic DKB/vocabulary lexicon matching (no LLM judge)
Systems	latency, tokens, peak memory; paired significance (t-test, Wilcoxon, bootstrap CI)	`scipy.stats`

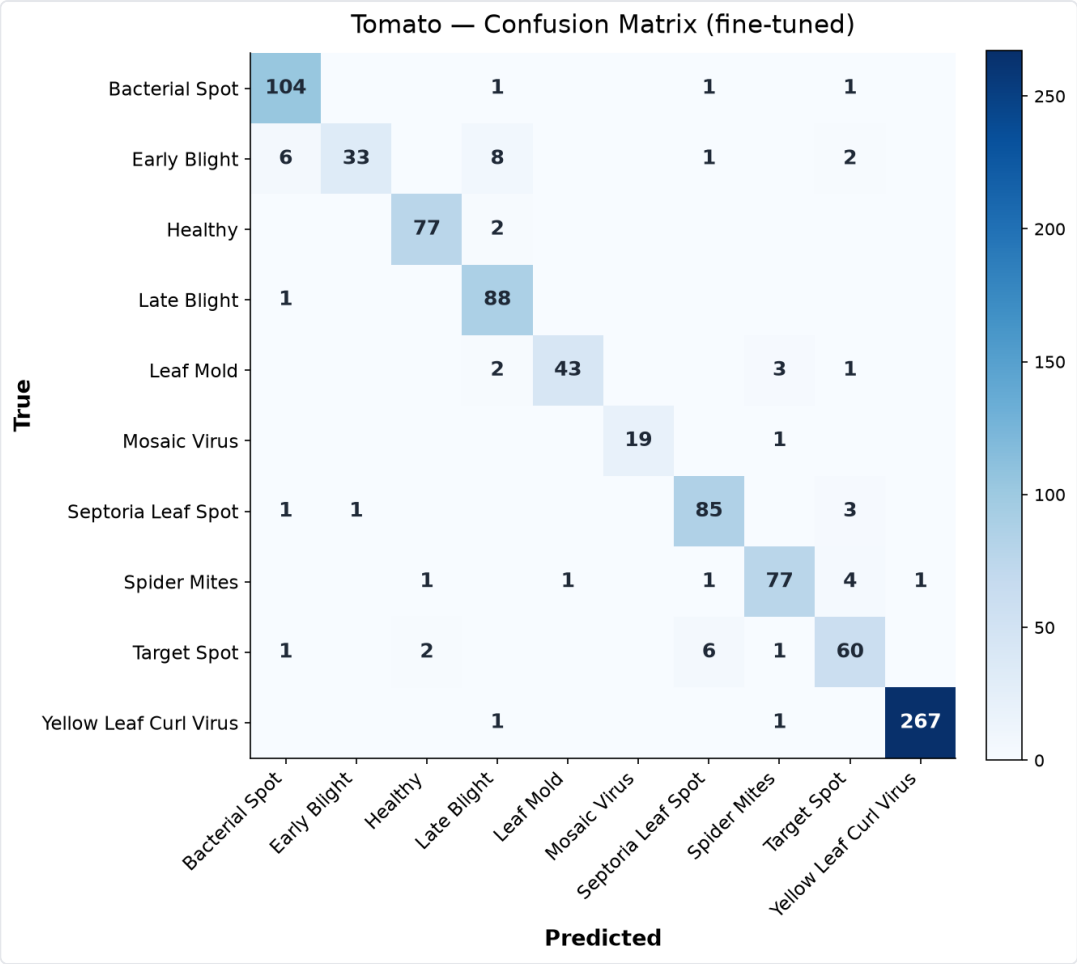
METRICS IN THE BRIEF THAT ARE NOT IMPLEMENTED

The brief lists **SPICE** and **CLIPScore**. Neither is implemented in the repository — there is **no evidence** of a SPICE or CLIPScore code path. Likewise, **no formal human-evaluation** protocol artifact exists (the `qa/` package is a stub). Error analysis is present in the form of confusion matrices, per-disease tables, and hallucination/clinical breakdowns — not a separate human-rated study.

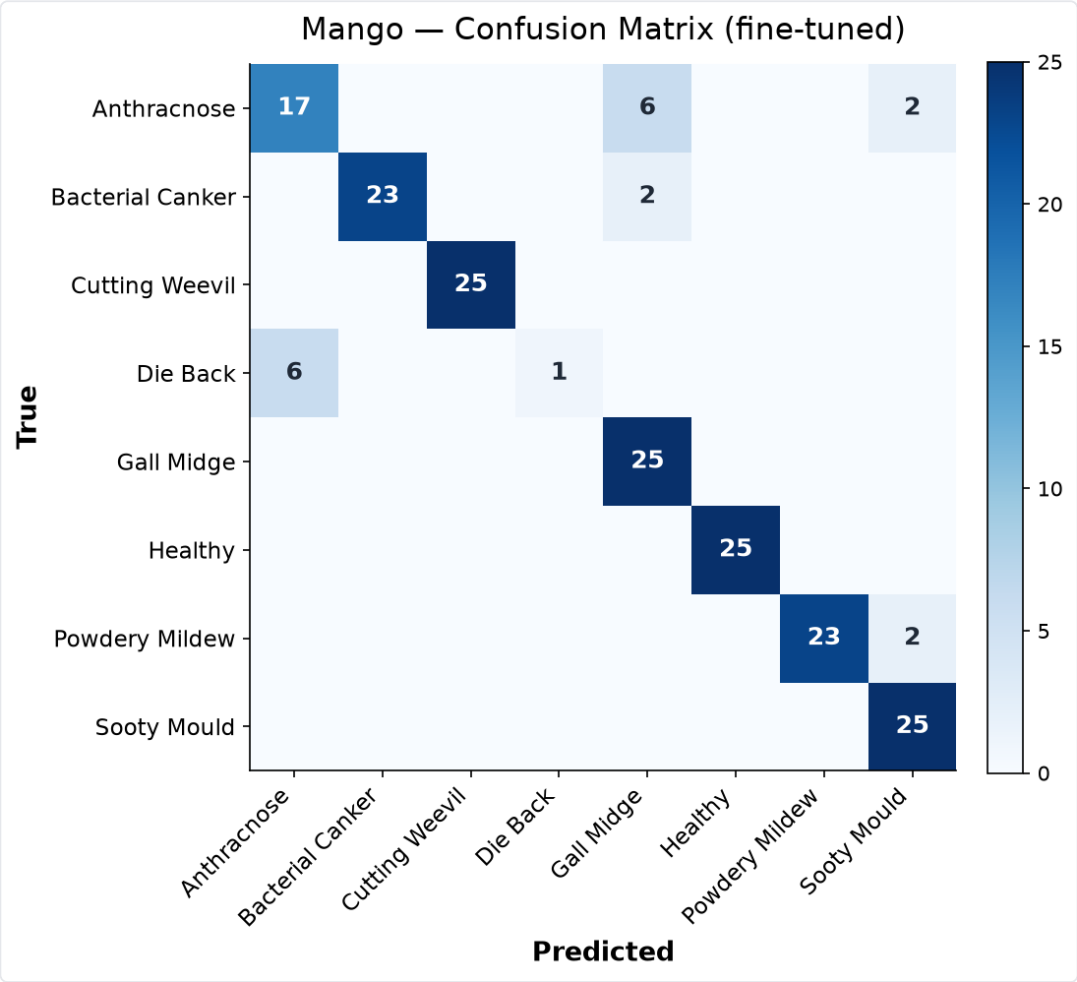
10.2 Headline results (real, from `metrics.json`)

Crop	Base acc	FT acc	Macro-F1	Weighted-F1	BLEU-4	ROUGE-L	CIDEr	BERTScore
Tomato (910)	7.3%	**93.7%**	0.034→**0.919**	0.025→**0.937**	0.003→**0.192**	0.093→**0.428**	0.000→**0.956**	0.843→**0.907**
Mango (200)	17.5%	**82.0%**	0.128→**0.811**	0.128→**0.811**	0.004→**0.190**	0.098→**0.423**	0.000→**0.946**	0.843→**0.907**

10.3 Confusion matrices (fine-tuned)



Tomato — strong diagonal; the main off-diagonal mass is Early Blight → Late Blight / Leaf Mold.



Mango — near-perfect except Die Back, which is almost entirely misread as other conditions.

EVIDENCE

Figures rendered by `scripts/render_readme_figures.py` directly from `reports/<run>/evaluation/confusion_matrix_finetuned.csv` — plotting the exact evaluated counts, recomputing nothing.

10.4 Error analysis — where the model is weakest

Crop	Weakest class	Accuracy	F1	Why (from the confusion matrix)
Tomato	Early Blight	66.0%	0.786	Confused with Late Blight / Leaf Mold (overlapping lesion appearance).
Mango	Die Back	4.0%	0.077	Single-leaf symptoms overlap heavily; hardest to name from a caption alone.

RESEARCH NOTE — DIE BACK IS HONEST, NOT HIDDEN

Mango Die Back scoring 4% is surfaced prominently rather than smoothed over. It reflects a genuine limitation: die-back's leaf-level appearance is not distinctive, and its DKB display name ("Die Back") diverges from the model's phrasing ("dieback"), which also stressed the classification parser (§12). The handbook records the weakness because hiding it would violate the project's honesty invariant.

10.5 How evaluation evolved: single-crop → crop-agnostic

Evaluation v1 (`v1.0-evaluation`) assumed tomato. Three independent hardcoded `"tomato"` literals — the default output directory, the label-map call, and a `_CROP` constant — meant that pointing evaluation at mango silently produced tomato labels. Evaluation v2 (`v0.2.0`) removed all three by deriving crop from the dataset manifest and the adapter's own run name. This is analyzed as a research challenge in §12.

> **EVIDENCE**
> Commit `673ed77`; `evaluation/config.py:resolve_crop()`, `evaluation/inference_runner.py` (crop-derived label map), `evaluation/report.py` (crop-derived lexicons + title). Regression tests in `tests/unit/evaluation/` pin the new contract.

10.6 Metric limitations, acknowledged

- **Classification via caption parsing** trades some recall (a paraphrased symptom may not match a verbatim DKB phrase) for full reproducibility — an "unclassified" bucket exists and is reported.
- **CIDEr** needs a multi-document corpus; on tiny `--max-samples` runs its TF-IDF is degenerate — expected behavior, documented.
- **Confidence from generation** (used in the demo) is a heuristic proxy for certainty, not a calibrated out-of-distribution detector.

11 · Engineering Challenges

The hardest problems in PlantDx were not algorithmic — they were **environmental**: MLX kernels, the numba/NumPy ABI, interpreter resolution, and memory. Each is presented as Problem → Root cause → Investigation → Solution → Lesson. Every one is traceable.

11.1 The M-RoPE VJP crash during QLoRA training

PROBLEM

A real QLoRA training run aborted with `[Primitive::vjp] Not implemented for CustomKernel`.

Root cause: the M-RoPE (multimodal rotary position embedding) Metal kernel in mlx-vlm 0.6.3 was a custom kernel without a vector-Jacobian-product (backward) implementation — fine for a forward pass, fatal for training, which needs gradients through it. **Investigation:** the crash traced into mlx-vlm's rotary-embedding path, confirming the missing VJP was in the kernel, not in PlantDx. **Solution:** upgrade to `mlx-vlm 0.6.4`, which wraps the M-RoPE kernel in `mx.custom_function` to provide the VJP (upstream PR #1475). Training then completed.

EVIDENCE

Concrete outcome: both adapters trained to completion on mlx-vlm 0.6.4 (`checkpoints/qwen25vl_<crop>_qlora/`). The environment sensitivity is why the training docs pin mlx-vlm and why DoRA is guarded (`docs/TRAINING.md`, `training/lora.py`).

LESSON

On a young framework (MLX/mlx-vlm), the *backward* path can lag the forward path by a version. Forward inference working is not evidence training will.

11.2 The numba / NumPy 2.x ABI conflict (BERTScore & the demo)

PROBLEM

`ImportError: numpy.core.multiarray failed to import` / `AttributeError: _ARRAY_API not found` — blocking BERTScore during evaluation and the whole model import in the demo, in one particular interpreter.

Root cause: a pre-existing environment issue, *not* a PlantDx bug. `bert-score` and `transformers` transitively import `librosa → numba`; an old numba build compiled against NumPy 1.x cannot run under the NumPy 2.x present in that interpreter. **Investigation:** reproduced in a clean venv to prove the conflict was environmental; confirmed the same code worked in a NumPy-2-compatible environment. **Solution:** (1) the evaluation `text_metrics.py` detects this specific failure and reports the exact fix (`pip install -U "numba>=0.59" "llvmlite>=0.42"`) instead of a raw traceback; (2) the demo degrades gracefully and instructs the user to launch with the working interpreter.

EVIDENCE

`docs/EVALUATION.md#troubleshooting`; `CHANGELOG.md` M6 entry ("a real, pre-existing environment bug found and documented, not patched-around silently"); commit `638b7bd` "stabilize evaluation dependencies and CI".

11.3 CI mypy failure — numpy dependency-group + stub-syntax

PROBLEM

GitHub Actions failed *only* on mypy: `Cannot find implementation or library stub for module named "numpy"`, while local mypy passed.

Root cause (two-layer): (1) `numpy` was declared only in an optional extra, but CI installs only `.[dev]` — local passed only because numpy happened to be present from other work; (2) once added with an unconstrained floor, pip resolved `numpy 2.5.0`, whose bundled `.pyi` stubs use PEP-695 `type X = …` syntax incompatible with mypy's `python_version = "3.10"`. **Investigation:** empirically bisected numpy versions (2.4.6 passes, 2.5.0 fails); reproduced in a fresh venv installing only `.[dev]`. **Solution:** pin `numpy>=1.26,<2.5` in `dev` (no `type: ignore`, no `ignore_missing_imports`).

LESSON

"Works locally" is not "works in CI" when CI installs a strict subset of dependencies. Reproduce in a fresh, minimal environment — the same discipline the repo already preached.

11.4 pytest could not even collect — circular import + basename collision

PROBLEM

For a long stretch, milestones were believed "pytest-clean" from ad-hoc scripts, but running the real `pytest` revealed it could not *collect*.

Root cause: (1) a circular import between `dataset/emitter.py` and `generation/engine.py`; (2) two identically-named test fixture files (`_dataset.py`) and two `test_discovery.py` in different directories, silently shadowing each other under pytest's default import mode. **Solution:** break the cycle with a `TYPE_CHECKING`-guarded import (zero runtime cost, since `from __future__ import annotations` already made the annotation lazy); rename the colliding files.

EVIDENCE

`CHANGELOG.md` "CI stabilization pass"; `CLAUDE.md §13.3` ("run the actual tool … a hand-written verification harness is not a substitute for the real CI gate").

LESSON (THE PROJECT'S OWN, VERBATIM IN SPIRIT)

"Verify with real tools, not reasoning." A custom verification script that never invokes `pytest` can hide a total collection failure.

11.5 The evaluation adapter-path bug (NotADirectoryError)

PROBLEM

`NotADirectoryError: …/adapters.safetensors/adapter_config.json` when evaluation received a `.safetensors` *file* path.

Root cause: mlx-vlm's `apply_lora_layers` requires the checkpoint *directory* (which holds `adapter_config.json` + `adapters.safetensors`), not the weights file. **Solution:** `resolve_adapter_dir()`

accepts either form, validates both required files exist, and raises a friendly `DerivationError` (never a raw traceback) for a malformed checkpoint — crop-agnostic, so it works for tomato, mango, or any future run.

EVIDENCE

`src/plantdx/evaluation/checkpoint.py:resolve_adapter_dir`; regression tests in `tests/unit/evaluation/test_eval_checkpoint.py`.

11.6 The Streamlit demo trilogy: wrong interpreter, segfault, degenerate confidence

The demo surfaced three distinct runtime problems, all recorded in `docs/DEMO_APP.md` and commit `088c134`:

Symptom	Root cause	Fix
Wall of numba tracebacks at launch	`conda activate` didn't change which `streamlit` / `python` ran (PATH still resolved to a broken interpreter)	Launch via the absolute interpreter path; disable Streamlit's file watcher; friendly banner; `BaseException` guard
Segmentation fault after several predictions	MLX Metal buffer cache grew unbounded in the long-lived process until 24 GB exhausted	Bound the MLX cache (`set_cache_limit`) and clear it after each run; cap model cache to one crop resident
Confidence always ≈1e-9	the one-shot `generate()` only exposes the *last* token's full-vocab logprobs; averaging over the whole vocabulary ≈ 0	iterate `stream_generate` and average the probability of the model's *own emitted* tokens → real ~0.85–0.94 confidence

LESSON — THE ENVIRONMENT IS PART OF THE PROGRAM

Three of the nastiest bugs were about *where and how* code ran (interpreter, GPU memory lifetime, telemetry semantics), not what it computed. A robust demo must treat the environment as an input to validate, not an assumption.

11.7 Aggregate: the stabilization tax

ENGINEERING NOTE

Roughly one commit in five exists purely to make the tree green without behavior change (`584f49e`, `638b7bd`, `3774492`). Across the build, ~120 Ruff findings and multiple mypy issues were resolved; two lint rules were *deliberately* suppressed with justified `# noqa` rather than "fixed," because the suggested fix would break a public API (`Qwen2_5VLConverter`, `InvariantViolation`). Stabilization was treated as first-class engineering, not chore-work.

12 · Research Challenges

Where §11 is about the environment, this section is about **wrong assumptions and unexpected results** — the research missteps that changed the project. The user rightly flags this as extremely important; it is where the real learning lives.

12.1 The wrong assumption that created the audit engine

Assumption: "the tomato dataset is a filtered 10-class tomato set." **Reality:** it was the full 38-class PlantVillage. **What changed:** a Normalization engine became mandatory, and "audit your data before architecting on it" became a project principle. This is documented as §6; it belongs here too because it was a *research* error (an unverified premise) with an engineering consequence.

12.2 The most instructive bug: hardcoded crop in evaluation

UNEXPECTED RESULT

After training mango successfully, evaluating it produced **tomato** labels, reports written to the **tomato** directory, and ~12.5% accuracy (random for 8 classes) — despite inference clearly running on mango images.

Root cause (three independent hardcoded literals): (1) the default output directory was a fixed `"…tomato_qlora…"` string; (2) inference called `load_label_map("tomato")`; (3) the analysis module had a `_CROP = "tomato"` constant. Because tomato's label map lacked mango's classes, ground-truth ids became `unknown:anthracnose`; and because "healthy" exists in *both* label maps, mango's healthy images were mislabeled `tomato_healthy` — the source of the spurious 12.5%. **Conclusion:** the fix was not "replace tomato with mango" but to make crop a *derived* value — read from `<dataset>/manifest.json`, recorded into stage 1's metadata, and consumed by stage 2. **Impact:** the pipeline became truly crop-agnostic; adding a third crop needs no evaluation-code change.

EVIDENCE

Commit `673ed77`; `evaluation/config.py:resolve_crop()`; the frozen mislabeled run's own `predictions.jsonl` (with `disease_id: "unknown:anthracnose"`) preserved the symptom before the fix. Regression tests added in `tests/unit/evaluation/`.

LESSON — A SECOND DATA POINT FALSIFIES HIDDEN ASSUMPTIONS

Tomato worked "by accident": every hardcoded default coincidentally matched reality. Only a second crop revealed that the code had no concept of "crop" at all. The generalization was forced by mango, not foreseen.

12.3 The classification parser vs. real model phrasing

UNEXPECTED RESULT

Valid-looking captions returned `unclassified`: "This mango leaf shows mango dieback" did not match the DKB's `"Die Back"`; "sooty mold" did not match `"sooty mould"`.

Root cause: the frozen extractor matches exact lowercased DKB phrases — brittle against one-word variants, spelling (US/UK), acronyms, and punctuation. **Conclusion:** do *not* weaken the frozen production extractor (it defines the evaluation numbers); instead add an app-layer wrapper that first calls the

production extractor and, only on `unclassified`, retries with normalized/alias, crop-scoped matching. **Impact:** the demo is robust without changing evaluation semantics.

EVIDENCE

`app/classification.py` (10 parametrized cases including dieback, sooty mold, TYLCV, punctuation, and correct rejection of terse/gibberish); it wraps `plantdx.evaluation.classification.extract_disease_id` unchanged.

12.4 Out-of-distribution field photos

UNEXPECTED RESULT

Real internet/field photos of septoria-looking leaves were misclassified (as TYLCV, early blight, or unknown), despite 93.7% test accuracy.

Root cause: the model was trained and evaluated only on PlantVillage-style single-leaf studio images; casual phone/field photos (a hand, cluttered background, age-yellowing) are out-of-distribution. **Verification:** feeding eight *PlantVillage* septoria images returned 8/8 correct — same model, same code — isolating the cause to image distribution, not a bug. **Conclusion:** surface the limitation honestly (low-confidence / unknown states) rather than asserting a confident wrong answer; making the model robust to field photos would require retraining on more diverse data (out of scope).

LESSON — REPORT THE DISTRIBUTION, NOT JUST THE SCORE

93.7% is a real in-distribution number and an unreliable out-of-distribution promise. The honest move is to expose the gap in the product, not to hide it behind a headline metric.

12.5 A descoped research ambition

RETROSPECTIVE

The original evaluation design imagined a **four-model zero-shot comparison matrix** (`ZeroShotEvaluator`, `ComparisonReporter`). That scope was narrowed to a single base-vs-fine-tuned comparison for the one trained model. This was a deliberate, documented descope — not a failure — but it is a research ambition that remains unrealized, and the handbook records it as such rather than implying four models were compared.

13 · Major Refactors

PlantDx refactored in one direction throughout its history: **smaller, more explicit, less clever.** Every review pass pushed that way. The refactors below are the ones with lasting structural impact.

13.1 The post-review simplification (v0.5)

Before	After	Reason / trade-off
`core/interfaces.py` — generic Protocol layer	deleted; concrete ABCs kept	Nothing consumed it; the ABCs sufficed. Less indirection.
`SeedDeriver` class	`image_seed` / `caption_seed` / `attempt_seed` pure functions	No state to carry; functions are clearer and testable.
7-file `dataset/converters/` package	one `dataset/converters.py`	Seven one-line stub files were over-scaffolding; registry API unchanged.
`dataset/schema.py`	`dataset/serialization.py`	Disambiguate from `config/schema.py`.

> **LESSON EMBEDDED IN THE CODEBASE'S OWN HISTORY**
>
> "It is easier and cheaper to consolidate over-abstraction after the fact than to prevent every instance of it up front — but prefer not needing to." The v0.5 pass is the concrete instance of that principle being applied.

13.2 The "re-founding" refactors (ontology, vocabulary)

Rather than implement the M1 stubs (which depended on a caption-concept model that did not exist), fresh `domain/` subpackages were written against the real substrate (the compiled ontology). The old stubs were kept, not edited. **Trade-off:** two "ontology"/"vocabulary" names in the tree (a documented naming collision) in exchange for not forcing a wrong contract.

13.3 The evaluation crop-generalization refactor

Before: three hardcoded `"tomato"` literals. **After:** a single `resolve_crop()` reading the dataset manifest, plus `run_name_from_adapter_path()` deriving the output directory. **Impact:** crop-agnostic evaluation; the highest-leverage refactor in the project because it turned a whole class of "silently wrong for the next crop" bugs into an impossibility (§12).

13.4 The demo confidence refactor

Before: confidence from the one-shot `generate()` (degenerate ~1e-9). **After:** iterate `stream_generate` and average selected-token probability. **Trade-off:** one extra layer of telemetry parsing in exchange for a confidence signal that is actually meaningful and can drive the threshold / unknown logic.

13.5 The Streamlit API modernization

Before: six `use_container_width=True` calls (deprecated for removal after 2025-12-31). **After:** `width="stretch"`; requirement floor bumped to `streamlit>=1.59`. **Trade-off:** a higher minimum Streamlit in exchange for zero deprecation warnings and future-proofing.

EVIDENCE

`CHANGELOG.md` ("post-review simplification"); commits `584f49e`, `673ed77`, `088c134`, `9f08d99`; `app/requirements.txt`.

14 · Performance Improvements

Performance work concentrated on **fitting a 7B VLM into 24 GB and keeping a long-lived process stable** — measured, not estimated.

14.1 Making training fit at all — quantization + LoRA + memory tricks

Technique	Effect
4-bit quantized base (QLoRA by construction)	7B model fits; only 40.4M adapter params train (~0.5%)
Micro-batch 1 + grad accumulation 8	Effective batch 8 at ~8–14 GB peak (of 24 GB)
Gradient checkpointing	Trades compute for memory (recommended at 24 GB)
Image resize 448×448	Bounds visual-token count and peak memory (Qwen2.5-VL is otherwise dynamic-res)

14.2 Inference & demo — load once, stay flat

Metric	Measured	Mechanism
Model + adapter load	~3 s, once per crop	`st.cache_resource(max_entries=1)` — persists across reruns; predictions never reload
Inference latency	~0.8–1.7 s / image (warm)	Greedy decoding, warm model
Active memory	~5.8 GB, flat over many runs	`mx.set_cache_limit(4 GB)` + `mx.clear_cache()` after each run
Crop switching	evicts the other 7B model	`max_entries=1` keeps one crop resident

EVIDENCE

Measured directly via `mlx.core.get_active_memory()` across an 8-inference stress run (memory flat at 5.80 GB, no segfault); `app/inference.py` (`_configure_mlx_memory`, `_release_mlx_cache`, `get_model_handle`); `docs/DEMO_APP.md` performance section.

14.3 The segfault, before and after

```
BEFORE (generate + no cache bound):     AFTER (stream_generate + cache bounded/cleared):
  run 1  active ↑                          run 1  active 5.80 GB
  run 2  active ↑↑                         run 2  active 5.80 GB
  run 3  active ↑↑↑                        …
  …      → 24 GB exhausted → SEGFAULT      run 8  active 5.80 GB  (no crash)
```

14.4 Determinism as a performance property

Content-hashing and `lru_cache` (on DKB loads, adapter info, phrase indices) mean repeated compiles and repeated demo reads are cheap and byte-identical — reproducibility and speed reinforcing each other rather than trading off.

15 · Lessons Learned

The most durable lessons are the ones the project wrote into its own memory and then had to obey.

15.1 Engineering

- **Verify with real tools, not reasoning.** A hand-written check that never runs `pytest` hid a total collection failure for milestones (§11.4).
- **"Works locally" ≠ "works in CI."** CI installs a strict dependency subset; reproduce in a fresh, minimal environment (§11.3).
- **The environment is an input.** Interpreter resolution, GPU memory lifetime, and kernel version caused the worst bugs — not the algorithms (§11).
- **Fail closed, with friendly messages.** Every compiler aborts on any rule violation and writes no artifact; runtime errors resolve to actionable text, never a raw traceback.

15.2 Machine learning

- **Forward working ≠ backward working** on a young framework (the M-RoPE VJP, §11.1).
- **Report the distribution, not just the score.** 93.7% in-distribution says nothing about field photos (§12.4).
- **Telemetry has semantics.** "logprobs" meant the last token's full vocab, not per-token confidence — reading it naively produced a meaningless metric (§11.6).
- **QLoRA can be "free."** With a 4-bit base, LoRA-over-quantized-base *is* QLoRA — the efficiency was a consequence of the model choice, not a separate cost (§3.3).

15.3 Research & data

- **Audit data before architecting on it** (§6.2 — the "tomato" folder).
- **Real data beats fixtures.** RC1's three grammar bugs only appeared against the real DKB (§7.3).
- **A second data point falsifies hidden assumptions.** Mango revealed the code had no concept of "crop" (§12.2).
- **Ground truth belongs in one reviewable file.** Centralizing scientific truth in the DKB makes "is this correct?" answerable by a domain expert reading one JSON.

15.4 Project management, git, documentation

- **Scaffold the contract first, fill it in later** — but be willing to delete over-scaffolding (§4, §13).
- **Stabilization is first-class work**, not chore-work (~1 in 5 commits, §11.7).
- **Two documents, two audiences:** a short public `PROJECT_CONTEXT.md` and a long private engineering memory (`CLAUDE.md`) — the latter is why this handbook could be reconstructed at all.
- **Commit messages are not the only record.** With single-line early commits, the CHANGELOG and memory doc carried the "why" — a deliberate, if unusual, choice.

THE META-LESSON

Nearly every lesson reduces to one stance: *make the property you care about a mechanically-checked invariant, then let the machine enforce it.* Grounding, determinism, observability, severity honesty, crop-agnosticism — each became a validator, a golden hash, or a derived value rather than a promise.

16 · If We Started Again Today

16.1 What would remain (it worked)

- The **DKB-first, deterministic-compiler** spine — content-hashed, fail-closed. This is the project's identity and its strongest idea.
- **Crop derived from the dataset manifest** from day one (learned the hard way via mango).
- **Two-stage evaluation** communicating only through a frozen `predictions.jsonl`.
- **QLoRA on a 4-bit base via MLX** for local, reproducible training on Apple Silicon.
- **The engineering-memory doc** — the single highest-leverage artifact for continuity.

16.2 What would change

- **Bake crop-genericity in from the start.** Never hardcode a crop; the three-literal bug would never have existed (§12.2).
- **Pin the ML environment explicitly and early** — a lockfile plus a documented, single "known-good" interpreter, to pre-empt the numba/NumPy and mlx-vlm-version episodes (§11.2, §11.1).
- **Run the real CI gate from commit one.** The "pytest couldn't collect" period was avoidable (§11.4).
- **Design confidence and OOD handling before the demo**, knowing the one-shot logprobs are degenerate (§11.6).
- **Author crop-worded instruction banks up front** rather than discovering the tomato-worded prompt was being reused for mango (§7.2).

16.3 Experiments to skip, repeat, or add

Skip	Repeat	Add
Assuming a public dataset is pre-filtered (audit instead).	The zero-shot base measurement that motivates the whole approach.	The four-model comparison the `ZeroShotEvaluator` was scaffolded for.
Any reliance on ad-hoc verification scripts over real CI.	RC1-style hardening against real data, not fixtures.	An OOD / field-photo evaluation set to quantify the distribution gap (§12.4).

> **HONEST COUNTERFACTUAL**
>
> The single change with the largest expected payoff is **crop-genericity from the start**: it would have eliminated the most instructive bug in the repository and made the second crop a pure config addition — which, notably, is exactly what mango eventually became (six files) once the generalization landed.

17 · Interview Appendix

Every question below is answerable from facts in this repository — no fabricated scenarios. They are the questions a rigorous reviewer would ask about *this* project. Pointers to the answering section are given so the handbook doubles as a study guide.

17.1 Research

1. Why does PlantDx forbid using a VLM to author training captions, and what evidence justifies it? (§1.3, §3.5)
2. Why captioning rather than classification, given classification metrics are still reported? (§3.1)
3. What does "no hallucination by construction" mean, and which mechanisms enforce it? (§3.9, §7)
4. Why an intermediate ontology instead of consuming DKB prose directly downstream? (§3.7)
5. What is "severity honesty" and why is it a research position, not a style choice? (§7.4)

17.2 Machine learning

1. Why is this QLoRA "by construction," and how many parameters actually train? (§3.3, §8.1 — 40.4M, rank 16)
2. What does `train_on_completions=true` change about the loss? (§9.4)
3. The base model scores 7.3%/17.5%; the fine-tuned 93.7%/82.0%. What does the confusion matrix say about *where* it still fails? (§10.3–10.4)
4. Why is the demo's confidence computed from `stream_generate` and not `generate`? (§11.6)
5. Why do good test numbers not transfer to field photos, and how was that isolated? (§12.4)

17.3 Python / backend / architecture

1. How does the three-layer training config compose, and how is it validated? (§9.1, §5.6)
2. Why do the two evaluation stages never share memory, and what is their only contract? (§5.4)
3. How is determinism made a *checked* property rather than an aspiration? (§3.8 — golden hashes)
4. Why are several packages intentionally left as stubs instead of deleted? (§4.3)
5. How does `st.cache_resource(max_entries=1)` keep the demo warm and memory-bounded? (§14.2)

17.4 Experiments & trade-offs

1. Why grad-accumulation 8 with micro-batch 1 rather than a larger batch? (§9.3, §14.1)
2. What did RC1 change, and why did those bugs only appear against the real DKB? (§7.3)
3. Classification via caption parsing trades recall for what, and how is the trade surfaced? (§10.6 — the "unclassified" bucket)
4. Why is CIDEr computed over the full batch, and when is it degenerate? (§10.1, §10.6)

17.5 Debugging

1. Walk through the crop-hardcoding bug: three literals, the 12.5% artifact, and the fix. (§12.2)
2. Diagnose `[Primitive::vjp] Not implemented for CustomKernel` from first principles. (§11.1)
3. Why did `conda activate` not fix the demo's numba crash, and what did? (§11.6)

4. The CI-only mypy failure: why did local pass, and why did pinning numpy fix it? (§11.3)
5. How was the "field photo is misclassified" complaint proven to be distribution, not a bug? (§12.4)

NOTE ON SCOPE

Questions about the three declared-but-untrained models (Qwen3-VL, InternVL3, Gemma-3) or about Molmo/Pixtral/Florence should be answered as: not trained / not present in this repository (§8). A truthful "we did not do that yet" is a stronger answer than an invented one.

18 · Complete Retrospective

This is the whole arc in one narrative — every beat backed by an artifact.

The seven-day arc

The empty repository (2026-07-11, 12:12 UTC). PlantDx began not with a model but with a conviction: general-purpose VLMs cannot be trusted to diagnose crop disease zero-shot, so they must not author the captions that teach a student to. The first commit encoded that conviction as *structure* — a fully-typed package scaffold with every stage stubbed, so the shape of the answer existed before any answer did.

The first experiment and the first surprise. The first real code was an audit engine, and it immediately falsified the project's most basic premise: the "tomato" dataset was the entire 38-class PlantVillage, ~54k images. A Normalization engine was born from that finding — the first time the data reshaped the architecture rather than the reverse.

Building the knowledge spine. Over Day 2–3 the deterministic compiler chain took shape: DKB → ontology (574 concepts, content-hashed) → vocabulary (268 items) → concept models → templates → a validated caption corpus. Each stage a pure function, each output a golden hash. The corpus was hardened (RC1) against the real DKB, catching grammar failures fixtures had hidden, and frozen at 1,966 captions.

First training. The tomato QLoRA pipeline completed on Day 3 — a config-driven orchestration of mlx-vlm, QLoRA "by construction" over a 4-bit base, 40.4M adapter parameters, on a 24 GB Mac. Training was not frictionless: a missing backward kernel in mlx-vlm 0.6.3 aborted the run until the 0.6.4 upgrade supplied the M-RoPE VJP.

First evaluation, first breakthrough number. Day 4 delivered the two-stage evaluation framework and the result that validated the entire thesis: tomato accuracy went from **7.3%** → **93.7%**. The base model — the very kind of VLM the project refused to trust with captions — was transformed by supervision it could never have written for itself.

The second crop, and the most instructive bug. Mango arrived as almost pure config (six files). Evaluating it exposed three hardcoded "tomato" literals that had made tomato work by accident; mango was scored against tomato's labels and reported ~12.5% (random). The fix — derive crop from the dataset manifest — generalized the whole pipeline and is the project's clearest lesson in how a second data point falsifies invisible assumptions. Mango then reached **82.0%** (base 17.5%).

From pipeline to product. After a short pause, the final three commits turned the trained adapters into a usable Streamlit demo — with a real confidence signal (the last-token-logprob trap avoided via `stream_generate`), adapter verification, organized uploads, and honest out-of-distribution handling — then overhauled the README with real results and figures and polished the whole thing for public release (`v1.0.0`).

Current state

An end-to-end, deterministic, crop-agnostic pipeline: cited knowledge base → grounded caption corpus → QLoRA fine-tuning of Qwen2.5-VL on Apple Silicon → rigorous evaluation → interactive demo, with CI green (`ruff` / `mypy` / `pytest`) throughout. Two crops trained and evaluated with real, reproducible numbers. Three more models and the image-grounded builder are scaffolded and awaited.

THE THROUGH-LINE

PlantDx is, at bottom, one idea executed with unusual discipline: *if you want a property to hold, make it impossible to violate.* Grounding is a validator. Reproducibility is a golden hash. Observability is a graph edge. Crop is a derived value. The project's real contribution is not a model that scores 93.7% — it is a method in which "the captions are grounded and the dataset is reproducible" is a fact about the software, checkable by anyone with the repository.

19 · Research Decision Log

The structured spine of the project. Each major decision is recorded as **Question → Hypothesis → Experiment → Result → Conclusion → Impact → Evidence**. This is what turns a code walkthrough into a research-engineering record: it captures not just what was built, but what was believed, tested, and learned.

RDL-01 · May a VLM author or judge the training captions?

QUESTION

Can an off-the-shelf VLM be trusted to write the grounded captions that supervise fine-tuning?

HYPOTHESIS

No — a model that cannot diagnose reliably will not describe reliably, and its errors would be inherited by the student.

EXPERIMENT

Measure the base VLM zero-shot on the held-out test splits.

RESULT

7.3% (tomato) / 17.5% (mango) accuracy — near chance for the class counts.

CONCLUSION

Ban models from the caption path entirely; ground captions in a cited knowledge base instead.

IMPACT

Every downstream design (DKB, ontology, deterministic generator) follows from this. It is the project's founding constraint.

EVIDENCE

`reports/<run>/evaluation/metrics.json` (base block); `caption_framework/07`; `CLAUDE.md §1`.

RDL-02 · Captioning or classification?

QUESTION

Should the model output a label, or a grounded description from which a label is extracted?

HYPOTHESIS

A grounded caption carries the observable evidence that justifies the label — which is exactly what a knowledge-grounded system can verify; a bare classifier has nothing to ground.

EXPERIMENT

Build the pipeline to generate captions and derive classification by parsing them.

RESULT

Fine-tuned captions yield 93.7% / 82.0% classification (via extraction), plus strong caption-quality metrics.

CONCLUSION

Captioning is the right target; classification is a downstream read-out.

IMPACT

Shaped the entire language layer (concepts → templates → corpus) and the evaluation (extraction-based classification).

EVIDENCE

`concepts/`, `corpus/`, `evaluation/classification.py:extract_disease_id`.

RDL-03 · Full fine-tuning, or parameter-efficient (LoRA/QLoRA)?

QUESTION

How to fine-tune a 7B VLM within 24 GB of unified memory?

HYPOTHESIS

Full fine-tuning will not fit; LoRA over a 4-bit base (= QLoRA) will, at a tiny parameter cost.

EXPERIMENT

Configure QLoRA (rank 16) over the 4-bit MLX Qwen2.5-VL and train.

RESULT

Training fits at ~8–14 GB peak; only 40,370,176 adapter params train (~0.5% of 7B).

CONCLUSION

QLoRA "by construction" is the correct efficiency choice; it fell out of the 4-bit model selection.

IMPACT

Made local training feasible at all; is the reason the whole artifact is self-contained.

EVIDENCE

`configs/lora/qlora.yaml` ; `configs/models/qwen25vl.yaml` ; adapter param count from `adapters.safetensors` .

RDL-04 · Cloud or local (MLX / Apple Silicon)?

QUESTION

Where should training and inference run?

HYPOTHESIS

Local MLX on the available M4 Pro is sufficient and better serves reproducibility (no keys, no external service, no data egress).

EXPERIMENT

Build the training on `mlx_vlm.lora` and inference on `mlx-vlm` .

RESULT

Both crops trained and served locally; inference ~0.8–1.7 s/image, ~5.8 GB steady.

CONCLUSION

Local Apple-Silicon MLX is a first-class target, not a fallback.

IMPACT

Determined the dependency stack, the two-environment split, and the demo's launch story.

EVIDENCE

`pyproject.toml [train]` ; `docs/TRAINING.md` ; `training/command.py` .

RDL-05 · Consume DKB prose directly, or compile an ontology?

QUESTION

Should downstream stages read the DKB's prose fields, or an intermediate typed layer?

HYPOTHESIS

Without a typed layer, every stage re-implements the observability and severity rules and silently drifts.

EXPERIMENT

Compile the DKB into a typed, evidence-linked graph with fail-closed validation.

RESULT

574 concepts / 1,289 edges; "observable?" and "licensed?" become single-implementation graph queries; content-hash `25ae0f6d…`.

CONCLUSION

The ontology is worth its weight; a full OWL/RDF reasoner is not (rejected as over-heavy).

IMPACT

One correct implementation of the core invariants; vocabulary/concepts/generation all build on it.

EVIDENCE

`ontology_design/01`; `ontology/domain/`; `test_domain_determinism.py`.

RDL-06 · Deterministic generation, or stochastic?

QUESTION

Should the caption corpus be reproducible byte-for-byte?

HYPOTHESIS

An artifact that cannot be independently rebuilt is a claim, not an artifact.

EXPERIMENT

Make every stage a pure function; content-hash outputs; add golden-hash regression tests.

RESULT

Four frozen hashes (ontology/vocabulary/concepts/corpus); rebuilds are byte-identical.

CONCLUSION

Determinism is a mechanically-checked property, not an aspiration.

IMPACT

Any change to a hash is a reviewed version event — protects the scientific claim.

EVIDENCE

`tests/unit/*/test_*_determinism.py`; hashes in `CLAUDE.md §18.4`.

RDL-07 · Single-crop, or crop-agnostic evaluation?

QUESTION

Should evaluation know about "crop," and if so, how?

HYPOTHESIS (INITIAL, WRONG)

Tomato-specific defaults are fine.

EXPERIMENT

Train mango and run the tomato-era evaluation on it.

RESULT

Mango scored ~12.5% (random) with tomato labels and tomato output paths — three hidden hardcoded literals.

CONCLUSION

Crop must be a *derived* value read from the dataset manifest, never hardcoded.

IMPACT

The most instructive fix in the project; adding a third crop now needs zero evaluation-code change.

EVIDENCE

Commit `673ed77`; `evaluation/config.py:resolve_crop()`; regression tests.

RDL-08 · How should the demo compute confidence?

QUESTION

What is a trustworthy per-prediction confidence?

HYPOTHESIS (INITIAL, WRONG)

The one-shot `generate()` logprobs are a usable confidence.

EXPERIMENT

Inspect `generate()` output; then switch to `stream_generate` and average selected-token probability.

RESULT

`generate()` exposes only the last token's full-vocab logprobs → ≈1e-9 (meaningless); `stream_generate` gives ~0.85–0.94 on correct predictions.

CONCLUSION

Confidence = mean probability of the model's own emitted tokens; below threshold → low-confidence/unknown.

IMPACT

Enabled honest OOD handling in the demo instead of a fake 0% bar.

EVIDENCE

`app/inference.py` (`run_inference`, `_selected_token_prob`); `docs/DEMO_APP.md`.

RDL-09 · Fix robustness in production classification, or in an app-layer wrapper?

QUESTION

The frozen extractor returns "unclassified" for valid variants ("dieback", "sooty mold"). Where to fix it?

HYPOTHESIS

Editing the frozen extractor would change the evaluation numbers; a wrapper preserves them.

EXPERIMENT

Add `app/classification.py` that calls the production extractor first, then a normalized/alias, crop-scoped fallback only on "unclassified".

RESULT

10/10 tricky cases pass; production behavior and evaluation numbers unchanged.

CONCLUSION

Robustness belongs in the demo layer; the evaluation contract stays frozen.

IMPACT

Separation of "measured" from "demoed"; no risk to reproducibility.

EVIDENCE

`app/classification.py`; `tests/unit/app/test_app_classification.py`.

RDL-10 · Freeze the corpus early, or harden it against the real DKB first?

QUESTION

Is the generator good enough to freeze after passing synthetic-fixture tests?

HYPOTHESIS

No — fixtures miss real phrasing; run against the real 18-condition DKB before freezing.

EXPERIMENT

The RC1 hardening pass on the real DKB.

RESULT

Three real grammar bugs found (clause-as-primary, number agreement, negation leak); corpus grew 1,070 → 1,966 with better diversity; severity rejections 17 → 0.

CONCLUSION

Harden against real inputs, extend fixtures to reproduce, then freeze.

IMPACT

A higher-quality, more diverse frozen corpus; a reinforced "real data beats fixtures" principle.

EVIDENCE

`CHANGELOG.md` RC1; corpus hash `1225eb86…`.

RDL-11 · Regenerate results, or re-render figures for presentation only?

QUESTION

The README figures were small/transparent (dark-mode-unreadable). Regenerate the evaluation?

HYPOTHESIS

Never recompute a metric for presentation; re-render from the existing CSVs at higher quality.

EXPERIMENT

`scripts/render_readme_figures.py` reads `reports/<run>/evaluation/*.csv` and renders larger-font, opaque-white figures.

RESULT

Publication-quality, dual-theme-safe figures plotting the exact evaluated values — no metric changed.

CONCLUSION

Presentation and results are strictly separated; figures are reproducible from data.

IMPACT

A trustworthy README where every number and figure traces to a report.

EVIDENCE

Commit `9f08d99`; `scripts/render_readme_figures.py`.

RDL-12 · Delete superseded stubs, or keep them as a frozen design surface?

QUESTION

What to do with M1 stubs superseded by real implementations?

HYPOTHESIS

Keeping typed stubs marks the frozen contract for future milestones (image grounding, other models) without re-design.

EXPERIMENT

Write fresh `domain/` packages next to the stubs; keep `generation/`, `dataset/`, `qa/`, `zero_shot` as typed stubs.

RESULT

Real code coexists with a stable future surface; a documented naming collision is the cost.

CONCLUSION

Keep intentional stubs; delete only genuine dead over-scaffolding.

IMPACT

M4/M5 (image grounding, three more models) can be filled in against fixed contracts.

EVIDENCE

`CLAUDE.md §4.2–4.3, §6.2`; `src/plantdx/{generation,validation,diversity,dataset,qa}/`.

Appendix — Evidence index & glossary

A · Primary evidence index

Artifact	What it grounds
`knowledge_base/dkb.json`	18 diseases × 46 fields, citations, target_models; the single source of truth.
`caption_framework/` (8 docs)	Frozen caption-pipeline design spec.
`ontology_design/` (13 docs)	Frozen ontology design spec.
`CHANGELOG.md`	Per-milestone rationale (the "why" for early commits).
`CLAUDE.md`	Detailed engineering memory (gitignored); the narrative backbone.
`configs/train,models,lora/*.yaml`	All training hyperparameters.
`checkpoints/<run>/adapter_config.json`	LoRA rank/scale/targets; adapter params.
`reports/<run>/evaluation/`	All metrics, confusion matrices, per-disease tables, figures.
`docs/{TRAINING,EVALUATION,DEMO_APP}.md`	Per-stage usage + troubleshooting (numba/NumPy, interpreter).
`tests/unit/*/test_*_determinism.py`	The four golden content hashes.

B · Frozen content hashes

Artifact	content_hash (prefix)
Ontology	`sha256:25ae0f6d9692a6d0…`
Vocabulary	`sha256:99d5833cf6405eea…`
Concepts	`sha256:e6f4870b777d6386…`
Corpus	`sha256:1225eb8605edb718…`

C · Glossary

Term	Meaning in PlantDx
DKB	Disease Knowledge Base — the cited, FINAL source of scientific truth.
Observability invariant	A caption may assert only what is visible on a single leaf.
Severity honesty	No per-image severity claim (the source data has none).
QLoRA "by construction"	LoRA over an already-4-bit base is QLoRA without extra steps.
Crop-agnostic	Crop is derived from the dataset manifest, never hardcoded.
V-ONT / V-VOC / V-CON / V-TPL / V-CAP	The fail-closed validator batteries at each stage.
Golden hash	A content hash pinned in a test; any change is a reviewed version event.
Stub	A typed, unimplemented package that marks a frozen future contract.

D · Provenance of this document

Compiled 2026-07-18 from the repository state at commit `9f08d99` (`main`). Sources: full git history (16 commits, 12 tags), the authenticated GitHub CLI (0 issues / 0 PRs / 0 releases), the in-repo specs and documentation, the evaluation reports, and the configuration files. Where evidence was thin or absent — the multi-model zero-shot benchmark, SPICE/CLIPScore, human evaluation, Molmo/Pixtral/Florence — the document says so explicitly rather than inventing. Every material claim is intended to be traceable to one of the artifacts named in its Evidence box.

PlantDx — Engineering & Research History · end of document.